

Горчак Дмитрий, 212. Двунправленный список в STL. Шаблоны, итераторы и упорядочивание объектов

Двунправленный (двусвязный) список – структура данных, в которой объекты не лежат в памяти подряд, а связаны ссылками: каждый знает своего соседа слева и соседа справа. В C++ такой список уже написан и лежит в стандартной библиотеке под именем `std::list`. Написан он как шаблон (`template` – образец): тип хранимых объектов в коде списка не указан, его подставляют при использовании.

Дальше по порядку: как устроен двунправленный список и что из этого устройства следует; как положить в него объекты и упорядочить их по разным признакам; как работают шаблоны, благодаря которым один и тот же список хранит и числа, и фигуры; и какое место список занимает среди остальных контейнеров STL. Примеры взяты из учебного проекта `cones`: программа читает из файла параметры фигур и создаёт точку, круг или конус в зависимости от того, сколько чисел в строке (два, три или четыре), а затем расставляет фигуры сначала по возрастанию площади основания, потом по возрастанию объёма.

Устройство двунправленного списка

Список состоит из заголовка и элементов. Заголовок хранит указатель на первый элемент и число элементов `n`. Каждый элемент хранит три вещи: сам объект, ссылку на предыдущий элемент и ссылку на следующий. Крайние элементы вместо ссылки на несуществующего соседа хранят пустой указатель.

[заголовок: $n = 3$, $first$] --> [Cone A] <--> [Cone B] <--> [Cone C]

Из этого устройства следуют все свойства списка:

- 1) по списку можно идти в обе стороны – от начала к концу и обратно; отсюда и название «двунправленный»;
- 2) вставка и удаление сводятся к переключению двух ссылок: соседние элементы не сдвигаются и не копируются, сколько бы их ни было;
- 3) обращения по номеру нет – чтобы добраться до пятого элемента, нужно пройти через четыре предыдущих;
- 4) на каждый элемент тратится дополнительная память под две ссылки;
- 5) адрес объекта не меняется при вставках, удалениях и сортировке – переставляются только ссылки, поэтому указатели на элементы остаются верными.

В стандартном `std::list` устройство то же, но список замкнут в кольцо через один служебный элемент: он стоит между последним и первым и играет роль «конца». Число элементов хранится в заголовке, поэтому `size()` выполняется мгновенно, а не пересчётом.

Создание списка и добавление объектов

Нужен заголовочный файл `<list>`. Список объявляется одной строкой, в угловых скобках указывается тип хранимых элементов. В одном списке лежат разные фигуры, поэтому хранятся не сами объекты, а указатели на общий базовый класс `Point` – так работает полиморфизм (способность одного вызова приводить к разным действиям в зависимости от настоящего типа объекта):

```

#include <list>
#include <memory>

typedef std::shared_ptr<Point> FigurePtr;    /* указатель на любую фигуру */

std::list<FigurePtr> figures;

for (std::size_t i = 0; i < loaded.size(); ++i) {
    figures.push_back(loaded[i]);    /* новый элемент в конец списка */
}

```

push_back добавляет в конец, push_front – в начало, front и back возвращают первый и последний элементы, size – число элементов, empty – пуст ли список. Ни одна из этих операций не зависит от длины списка.

Проход по списку: итераторы

Итератор (iterator – перебиратель) – объект, который указывает на элемент контейнера и умеет переходить к соседнему. Его разыменовывают (*it – получить сам элемент), сдвигают (++it) и сравнивают с другим итератором. Записи вида figures[3] у списка нет – перебор идёт только итератором:

```

for (std::list<FigurePtr>::const_iterator it = figures.begin();
     it != figures.end(); ++it) {
    (*it)->print(std::cout);    /* у каждой фигуры свой вывод */
}

```

const_iterator – итератор, через который элементы можно только читать. Звёздочка даёт сам элемент списка (указатель на фигуру), а стрелка вызывает метод объекта, на который он указывает. Диапазон задаётся парой итераторов [first, last): last указывает уже за последний элемент, поэтому цикл заканчивается по условию it != end(), а не it <= end().

Двунаправленность проверяется обратным проходом: тот же список читается от конца к началу обратным итератором, который получают методами rbegin и rend.

```

for (std::list<FigurePtr>::const_reverse_iterator it = figures.rbegin();
     it != figures.rend(); ++it) {
    std::cout << volume_of(*it) << ' ';
}

```

Итераторы разных контейнеров умеют разное. У вектора итератор произвольного доступа (random access): к нему можно прибавить число и сразу попасть на нужный элемент, потому что элементы лежат подряд. У списка итератор двунаправленный (bidirectional): он шагает только вперёд и назад, по одному элементу. Это различие объясняет следующий раздел.

Упорядочивание по площади и по объёму

Порядок задаётся функцией сравнения: она получает два элемента и возвращает true, если первый должен стоять раньше. Площадь и объём не хранятся в полях, а считаются методами; у точки площади нет, у точки и круга нет объёма, поэтому для них возвращается ноль:

```

bool less_by_area(const FigurePtr &left, const FigurePtr &right) {
    return area_of(left) < area_of(right);
}

```

```

bool less_by_volume(const FigurePtr &left, const FigurePtr &right) {
    return volume_of(left) < volume_of(right);
}

figures.sort(less_by_area);    /* порядок по площади основания */
figures.sort(less_by_volume);  /* порядок по объёму */

```

Вызывается не общий алгоритм `std::sort`, а собственный метод списка `sort`. Причина – в итераторах: `std::sort` прыгает по элементам туда-сюда, а итератор списка так не умеет, и попытка вызвать `std::sort` для списка просто не соберётся. Метод `list::sort` реализован сортировкой слиянием: он переставляет ссылки между элементами, не копируя сами объекты, работает за $n \log n$ и сохраняет взаимный порядок равных элементов (такую сортировку называют устойчивой).

Если список уже упорядочен, новый элемент можно сразу поставить на своё место. Поиск места – проход по списку, сама вставка стоит двух переключений ссылок независимо от длины списка:

```

void insert_sorted_by_volume(std::list<FigurePtr> &figures, const FigurePtr &value) {
    std::list<FigurePtr>::iterator place = figures.begin();

    while (place != figures.end() && volume_of(*place) < volume_of(value)) {
        ++place;
    }
    figures.insert(place, value);    /* insert ставит элемент перед place */
}

```

Удаление по условию делается одной командой: `remove_if` проходит список и выбрасывает подходящие элементы, не трогая остальные.

```

figures.remove_if(small_volume);    /* убрать фигуры с объёмом меньше порога */

```

Что получилось

Два порядка выходят разными. Заметнее всего это на конусе с широким, но низким основанием ($r = 2.80$, $h = 2.00$): по площади основания он предпоследний из девятнадцати фигур, а по объёму – только четырнадцатый. Круги в порядке по площади стоят наравне с конусами, а в порядке по объёму уходят в начало списка вместе с точками, потому что объёма у них нет.

--- Sorted by base area ---

15	Circle	12.50	17.50	2.00	-	12.57	-
16	Cone	19.00	4.00	2.20	9.00	15.21	45.62
17	Cone	14.00	12.00	2.40	8.00	18.10	48.25
18	Cone	3.00	5.50	2.80	2.00	24.63	16.42
19	Cone	9.00	8.00	3.00	10.00	28.27	94.25

--- Sorted by volume ---

14	Cone	3.00	5.50	2.80	2.00	24.63	16.42
15	Cone	8.00	17.00	1.70	5.50	9.08	16.65
16	Cone	15.00	15.50	1.80	6.50	10.18	22.05
17	Cone	19.00	4.00	2.20	9.00	15.21	45.62
18	Cone	14.00	12.00	2.40	8.00	18.10	48.25
19	Cone	9.00	8.00	3.00	10.00	28.27	94.25

Шаблоны: почему один список хранит что угодно

Шаблон – это код, написанный не для конкретного типа, а для типа, который будет подставлен позже. Объявление начинается словами `template <typename T>`, где `T` – имя-заполнитель. Простейший пример – функция, которая возвращает больший из двух объектов:

```
template <typename T>
T maximum(const T &left, const T &right) {
    return (left < right) ? right : left;
}

int a = maximum(3, 7);           /* T = int, тип выведен сам */
double b = maximum(2.5, 1.5);   /* T = double */
```

Компилятор по аргументам сам определяет, какой тип подставлен вместо `T`, и порождает отдельную функцию для каждого нужного типа. Это называется инстанцированием (*instantiation* – порождение экземпляра): в готовой программе лежат две разные функции, хотя в исходном коде написана одна. Отсюда требование: код шаблона должен быть виден там, где им пользуются, поэтому шаблоны почти всегда пишут прямо в заголовочных файлах.

Классы шаблонами бывают точно так же. Упрощённо двунаправленный список можно объявить так:

```
template <typename T>
class List {
public:
    struct Node {
        T value;           /* элемент списка */
        Node *prev;        /* сам объект */
        Node *next;        /* ссылка на предыдущий */
    };

    void push_back(const T &value);
    std::size_t size() const;

private:
    Node *first_;          /* заголовок: начало списка */
    std::size_t count_;    /* заголовок: число элементов */
};
```

Именно так устроен и `std::list`, только с проверками, работой с памятью и итераторами. Тип подставляется в угловых скобках при объявлении переменной, и из одного описания получаются совершенно разные списки:

```
std::list<int> numbers;      /* список чисел */
std::list<std::string> names; /* список строк */
std::list<FigurePtr> figures; /* список указателей на фигуры */
```

Требования к типу `T` нигде не перечислены: они возникают из того, что с ним делают. Если в шаблоне написано `left < right`, то подставленный тип обязан уметь сравниваться – иначе ошибка появится не при объявлении, а при инстанцировании. Поэтому у фигур порядок задан отдельной функцией сравнения: у указателя на `Point` своего смысла «меньше» нет.

Шаблонными бывают и отдельные методы, и параметры-числа (например, `std::array<double, 3>` – массив ровно из трёх чисел, размер известен на этапе компиляции). Общая идея одна: код написан один раз, а работает с любым подходящим типом, и при этом не теряется скорость – подстановка

происходит при сборке, а не во время работы программы.

Список среди остальных контейнеров

STL (Standard Template Library – библиотека стандартных шаблонов) – часть стандартной библиотеки C++, собранная из трёх согласованных частей: контейнеры (хранилища объектов), итераторы (единый способ их перебирать) и алгоритмы (готовые операции – сортировка, поиск, подсчёт). Контейнеры ничего не знают об алгоритмах, а алгоритмы – о контейнерах; связывают их итераторы, поэтому одна и та же сортировка работает с разными хранилищами.

Для каждой операции в стандарте указано, как растёт время её работы с увеличением числа элементов n : линейно (пропорционально n), за $\log n$ (почти не растёт) или за постоянное время. Выбор контейнера – это выбор того, какие операции окажутся дешёвыми.

Контейнер	Как устроен	Дёшево	Дорого
<code>vector</code>	динамический массив	доступ по номеру, добавление в конец	вставка и удаление в середине
<code>list</code>	двусвязный список	вставка и удаление в любом месте	нет доступа по номеру, поиск перебором
<code>forward_list</code>	односвязный список	минимум памяти на элемент	проход только вперёд
<code>deque</code>	блоки элементов	добавление с обоих концов	вставка в середину
<code>set</code> , <code>map</code>	сбалансированное дерево	поиск и вставка за $\log n$, лишняя память на связи	порядок
<code>unordered_set</code> , <code>map</code>	хеш-таблица	поиск за постоянное время	элементы не упорядочены

Отдельно стоят адаптеры (`adapter` – переходник): это не самостоятельные хранилища, а надстройки над контейнером, которые оставляют только нужные операции – `stack` (кладём и снимаем с одного конца), `queue` (кладём в конец, берём из начала), `priority_queue` (первым выдаётся наибольший элемент).

Список выгоден, когда элементы добавляются и удаляются по одному, а доступ к ним последовательный: фигуры создаются при чтении файла и потом упорядочиваются целиком. Вектор выгоден, когда нужен доступ по номеру и важна скорость перебора: его элементы лежат в памяти подряд, и процессор читает их заметно быстрее. Часто в одной задаче нужны оба: матрица попарных расстояний n на n – это вектор векторов с доступом по номерам, а очередь точек при обходе – адаптер `queue`.

Заключение

Готовый шаблон списка избавляет от написания своей структуры данных: достаточно подставить нужный тип и задать правило сравнения. Понимать устройство всё равно нужно – именно из трёх полей элемента (объект и две ссылки) следует, почему у списка своя сортировка, почему нет обращения по номеру и почему вставка в середину дешевле, чем в векторе.

*[объект создан] -(push_back)-> [элемент списка: объект + две ссылки]
[правило сравнения] -(list::sort)-> [порядок по площади или по объёму]
[итератор] -(++ / --)-> [проход вперёд и назад]*

Приложение. Памятка по `std::list`

Цель	Операция	Пример	Описание
Объявить список	<code>std::list<T></code>	<code>std::list<FigurePtr> figures;</code>	T – тип хранимых элементов
Добавить в конец	<code>push_back</code>	<code>figures.push_back(figure);</code>	новый элемент становится последним
Добавить в начало	<code>push_front</code>	<code>figures.push_front(figure);</code>	новый элемент становится первым
Вставить в середину	<code>insert</code>	<code>figures.insert(place, figure);</code>	ставит элемент перед итератором place
Удалить элемент	<code>erase</code>	<code>figures.erase(place);</code>	убирает элемент, на который указывает place
Удалить по условию	<code>remove_if</code>	<code>figures.remove_if(small_volume);</code>	убирает все подходящие элементы
Первый и последний	<code>front, back</code>	<code>figures.front()->print(out);</code>	возвращают сам элемент, а не итератор
Число элементов	<code>size, empty</code>	<code>figures.size();</code>	size выполняется мгновенно
Проход вперёд	<code>begin, end</code>	<code>for (it = figures.begin(); ...)</code>	end указывает за последний элемент
Проход назад	<code>rbegin, rend</code>	<code>for (it = figures.rbegin(); ...)</code>	обратный итератор, от конца к началу
Упорядочить	<code>sort</code>	<code>figures.sort(less_by_area);</code>	сортировка слиянием, $n \log n$, устойчивая
Перевернуть	<code>reverse</code>	<code>figures.reverse();</code>	меняет порядок элементов на обратный
Объединить списки	<code>merge, splice</code>	<code>a.merge(b);</code>	переносит элементы без копирования
Очистить	<code>clear</code>	<code>figures.clear();</code>	удаляет все элементы

Приложение. Памятка по шаблонам

Цель	Запись	Пример	Описание
Шаблон функции	<code>template <typename T></code>	<code>template <typename T> T maximum(...)</code>	тип выводится по аргументам
Шаблон класса	<code>template <typename T> class</code>	<code>template <typename T> class List</code>	тип задают в угловых скобках
Подставить тип	<code>Имя<Тип></code>	<code>std::list<int> numbers;</code>	порождается отдельный класс
Два параметра	<code>template <typename A, typename B></code>	<code>std::map<std::string, int></code>	параметров может быть несколько
Параметр-число	<code>template <typename T, std::size_t N></code>	<code>std::array<double, 3> v;</code>	размер известен при сборке
Где писать	в заголовочном файле	<code>list.h</code>	код нужен там, где шаблон используют